

React Native Assignment Submission Report

Candidate Sandesh Pawar

Position React Native Developer

Time Taken to Complete ~6 hours

Date _____

Project Restaurant Order Management App

1. Project Overview

The Restaurant Order Management App is a cross-platform mobile application built with

Expo SDK 54, React Native 0.81, React 19, and TypeScript. The application serves as an order management dashboard for restaurant staff to view, search, filter, and update the status of customer orders in real time. The app ships with 15 sample orders covering three status states — Pending, In Progress, and Completed — and persists all order data locally using AsyncStorage so that status updates survive app restarts.

2. Assignment Requirements Covered

- ☒ Login Screen
- ☒ Orders List (12+ Orders)
- ☒ Search
- ☒ Status Filter
- ☒ Order Details
- ☒ Update Order Status
- ☒ Local Persistence (AsyncStorage)
- ☒ TypeScript

3. Tech Stack

The following technologies and libraries were used to build the application:

Technology	Version	Purpose
React Native	0.81.5	Core framework for building native mobile UI
Expo	~54.0.35	Development platform and toolchain
TypeScript	~5.9.2	Static type checking and typed interfaces/enums
React Navigation	^7.3.14 / ^7.18.6	Native Stack navigation with typed routes
React Context API	Built-in	Feature-scoped state management for orders
AsyncStorage	2.2.0	Local data persistence layer
React Native Toast Message	^2.4.0	In-app success/error notifications
@expo/vector-icons	^15.1.1	Material Design icons throughout the UI
React Native Gesture Handler	~2.28.0	Touch handling and Pressable feedback
React Native Safe Area Context	~5.6.0	Safe-area aware layouts

4. Project Architecture

The project follows a feature-based modular architecture that separates concerns by domain. Each feature (auth, orders) is self-contained with its own screens, components, hooks, types, services, and data. Cross-cutting concerns such as navigation and shared UI (Toast) live at the top level of src/.

Folder structure:

```
src/
├── components/
│   └── Toast.tsx # Custom toast configuration
├── features/
│   ├── auth/
│   │   ├── constants/credentials.ts # Demo login credentials
│   │   └── screens/LoginScreen.tsx # Login UI with validation
│   └── orders/
│       ├── components/
│       │   ├── OrderCard.tsx # Reusable order list item
│       │   ├── SearchBar.tsx # Search input component
│       │   ├── StatusBadge.tsx # Status pill badge
│       │   └── StatusFilter.tsx # Filter chip group
│       ├── context/orderContext.tsx # React Context provider
│       ├── data/dummydata.json # 15 mock orders
│       ├── hooks/useOrder.ts # Custom hook wrapper
│       ├── screens/
│       │   ├── OrdersScreen.tsx # Orders list + search/filter
│       │   └── OrdersDetailsScreen.tsx # Order detail + status update
│       ├── services/orderService.ts # AsyncStorage CRUD layer
│       └── types/order.ts # TypeScript interfaces & enums
├── navigation/
│   ├── AppNavigator.tsx # Native Stack navigator
│   └── types/types.ts # Typed route params
└── App.tsx # Root component + providers
```

State Management Flow: The orders state is managed through a feature-scoped React Context. The OrdersProvider wraps the app root in App.tsx, and the custom useOrders() hook exposes the context to screens. Data flows:

OrdersScreen / OrdersDetailsScreen

↓ useOrders()

- ↓ OrdersContext (React Context API)
- ↓ orderService.ts (loadOrders / saveOrders)
- ↓ AsyncStorage (@restaurant_orders)

Navigation Flow: Login → (on success, replaces stack) → Orders → (tap card) → OrderDetails

5. Features Implemented

Login Screen

Located at `src/features/auth/screens/LoginScreen.tsx`. The login screen presents a branded card with a chef-hat logo, email and password inputs (with a show/hide password toggle), and a Sign In button. Credentials are validated against hardcoded demo values (owner@dummy.com / password123) defined in `src/features/auth/constants/credentials.ts`. On success, `navigation.replace('Orders')` replaces the login route so the user cannot navigate back to the login screen. Invalid credentials trigger an Alert. The layout uses `KeyboardAvoidingView` with platform-specific behavior to handle keyboard overlap.

Location: `src/features/auth/screens/LoginScreen.tsx`

Orders List (15 Orders)

Located at `src/features/orders/screens/OrdersScreen.tsx`. The Orders screen fetches all orders via the `useOrders()` hook and renders them in a `FlatList`. Each order is displayed as an `OrderCard` showing the order ID, customer name, total amount, item count, date, and a status badge. A loading state shows a branded spinner while data is being loaded from `AsyncStorage`. An empty state is shown when no orders match the current search or filter. Tapping a card navigates to the `OrderDetails` screen with the order ID as a route parameter.

Location: `src/features/orders/screens/OrdersScreen.tsx`

Search

Implemented as a reusable `SearchBar` component (`src/features/orders/components/SearchBar.tsx`) with a magnifying-glass icon and clear button. The search filters orders in real-time by customer name and order ID using a case-insensitive substring match. The filtering logic is memoized with `useMemo` in `OrdersScreen` to avoid unnecessary re-computation on every render.

Location: `src/features/orders/components/SearchBar.tsx`

Status Filter

Implemented as a reusable `StatusFilter` component (`src/features/orders/components/StatusFilter.tsx`) with four filter chips: All, Pending, In Progress, and Completed. The selected chip is highlighted with a purple background. Filtering is combined with search in a single memoized filter function, so both criteria are applied simultaneously.

Location: `src/features/orders/components/StatusFilter.tsx`

Order Details

Located at `src/features/orders/screens/OrdersDetailsScreen.tsx`. The details screen receives an `orderId` route parameter and looks up the order via `getOrderById()` from the context. It displays a summary card (order ID, status badge, customer name, date, item count, total), a `FlatList` of all order items with individual line totals, and a grand total at the bottom. A back button in the header returns to the Orders list. If the order is not found, a fallback message is shown.

Location: `src/features/orders/screens/OrdersDetailsScreen.tsx`

Update Order Status

Implemented in the OrderDetails screen. A 'Mark as ...' button is shown when the order status is not yet Completed. It follows a linear progression: Pending → In Progress → Completed. The `updateOrderStatus` function in `orderContext.tsx` maps the current status to the next status, updates the in-memory state, and persists the change to `AsyncStorage`. A success toast notification confirms the update. The button shows an inline `ActivityIndicator` while the update is in progress.

Location: `src/features/orders/context/orderContext.tsx:52`

Local Persistence (AsyncStorage)

Implemented in `src/features/orders/services/orderService.ts`. On first launch, the app loads orders from `AsyncStorage`; if no data exists, it seeds the store with `dummydata.json` and saves it. Every status update triggers `saveOrders()`, which serializes the updated orders array to JSON and writes it to `AsyncStorage` under the key `@restaurant_orders`. A `resetOrders()` utility is also provided for development.

Location: `src/features/orders/services/orderService.ts`

TypeScript

The entire codebase is written in TypeScript. Key typed artifacts include: the Order and OrderItem interfaces (src/features/orders/types/orders.ts), the OrderStatus enum with Pending/InProgress/Completed values, the RootStackParamList type for typed navigation routes (src/navigation/types/types.ts), and typed props on every component and hook. The project uses TypeScript ~5.9.2.

Location: src/features/orders/types/order.ts

6. Data Flow

Navigation and State Flow

Navigation Flow: The app defines a single Native Stack navigator (src/navigation/AppNavigator.tsx) with three screens: Login (initial route), Orders, and OrderDetails. When the user successfully logs in, navigation.replace('Orders') replaces the Login route, preventing back navigation to the login screen. From the Orders list, tapping an OrderCard calls navigation.navigate('OrderDetails', { orderId: item.id }), passing the order ID as a typed route parameter. The OrderDetails screen reads this parameter from route.params. The navigator uses slide_from_right animation and hides the default header (headerShown: false), with custom headers implemented in each screen.

State Flow: Order data is managed through a feature-scoped React Context. The OrdersProvider (src/features/orders/context/orderContext.tsx) is mounted at the root of App.tsx, making the orders state available to all child screens. On mount, the provider calls initializeOrders(), which invokes loadOrders() from orderService.ts. This function first checks AsyncStorage for a persisted @restaurant_orders key; if found, it parses and returns the stored JSON. If not found, it seeds AsyncStorage with the 15 orders from dummydata.json and returns that data. The context exposes three values: orders (the array), loading (a boolean for the initial fetch), getOrderById (a memoized lookup function), and updateOrderStatus (an async function that advances an order's status and persists the change). The useOrders() custom hook (src/features/orders/hooks/useOrder.ts) is a thin wrapper around useContext that provides a clean API to consuming components.

Persistence Flow: All order state changes are persisted to AsyncStorage immediately. When updateOrderStatus is called, it maps over the orders array, advances the matching order's status (Pending → In Progress → Completed), calls setOrders to update React state, and then calls saveOrders() which serializes the

array to JSON and writes it to AsyncStorage. On the next app launch, loadOrders() reads from AsyncStorage first, ensuring status changes survive restarts.

7. Challenges & Solutions

- **AsyncStorage seeding on first launch**

Problem: On first launch, AsyncStorage is empty, so the app needs to populate it with initial data. Solution: loadOrders() checks if stored data exists; if not, it seeds AsyncStorage with dummydata.json and returns the data. This ensures subsequent launches read from local storage.

Location: src/features/orders/services/orderService.ts:9

- **Status progression logic**

Problem: Orders should only advance forward through the status lifecycle (Pending → In Progress → Completed) and should not regress. Solution: The updateOrderStatus function in orderContext.tsx uses a switch statement to map each current status to its next state, and the OrderDetails screen only shows the update button when the status is not Completed.

Location: src/features/orders/context/orderContext.tsx:52

- **Combining search and filter without performance issues**

Problem: Filtering and searching must work together efficiently without re-computing on every keystroke. Solution: Both search text and selected status are combined into a single useMemo call in OrdersScreen, so the filtered list is only recomputed when orders, searchText, or selectedStatus changes.

Location: src/features/orders/screens/OrdersScreen.tsx:32

- **Typed navigation with route parameters**

Problem: Passing orderId from the Orders list to the OrderDetails screen requires type safety. Solution: The RootStackParamList in src/navigation/types/types.ts defines OrderDetails as { orderId: string }, and NativeStackScreenProps provides type-safe access to route.params in the screen component.

Location: src/navigation/types/types.ts

- **Loading and empty states**

Problem: The app needs to handle async data loading gracefully and show a meaningful message when no orders match the current filters. Solution: OrdersScreen shows a branded loading spinner while loading is true, and

FlatList's ListEmptyComponent renders a 'No orders found' message with a suggestion to adjust search or filters.

Location: src/features/orders/screens/OrdersScreen.tsx:45

- **Callback memoization in Context**

Problem: Passing new function instances in the context value on every render would cause unnecessary re-renders of consumer components. Solution: getOrderById and updateOrderStatus are wrapped in useCallback with proper dependency arrays, and the context value is memoized with useMemo.

Location: src/features/orders/context/orderContext.tsx:45

8. AI Usage

The following AI tools were used during development:

- ChatGPT (OpenAI)
- Used for high-level architecture planning, navigation and state management strategy discussions, code generation and refinement for login screen, orders context, AsyncStorage service, screens, and reusable components, and technical guidance on React Context API, AsyncStorage integration, React Navigation setup, TypeScript typing, and component organization.
- GitHub Copilot
- Used as an inline coding assistant for generating boilerplate code, suggesting TypeScript type definitions, and providing real-time code completions during development.

Developer Responsibility: All AI-generated code was reviewed, integrated, tested, and modified as needed by the candidate before inclusion in the final submission. The candidate retains full responsibility for the correctness and quality of the final codebase.

9. Screenshots

Login Screen

5:06

651 KB/s       87 



Welcome Back

Sign in to manage your restaurant



Email address



Password



Sign In

 **Demo Credentials**

Email: owner@dummy.com

Password: password123

Orders List

37.7 KB/s

Completed

Completed

30/7/2026

Completed

30/7/2026

In Progress

30/7/2026

Pending

7 items

Order Details

5:03

26.6 KB/s



Order Details

#ORD001

Completed

Rahul Sharma

Date	Items	Total
30/7/2026	9 items	₹710.00

Order Items

Paneer Tikka	₹180.00
x2	₹360.00

Butter Naan	₹50.00
x3	₹150.00

Mango Lassi	₹80.00
x2	₹160.00

Roasted Papad	₹20.00
x2	₹40.00

Order Details with Toast

95.1 KB/s

Order #ORD003 marked as Completed

Completed

Date	Items	Total
30/7/2026	11 items	₹570.00

Order Items

Jeera Rice	₹140.00
x1	₹140.00

Masala Papad	₹30.00
x2	₹60.00

Rasgulla	₹30.00
x3	₹90.00

Pending Orders Filter

5:02

5.75 KB/s

Search by customer or order ID

All

Pending

In Progress

Completed

#ORD004

Sneha Gupta

Pending

Total

₹250.00

7 items

30/7/2026

#ORD008

Pooja Joshi

Pending

Total

₹690.00

6 items

30/7/2026

#ORD012

Meera Iyer

Pending

Total

₹400.00

9 items

30/7/2026

#ORD015

Arjun Menon

Pending

Total

7 items

In Progress Filter

5:02

5.75 KB/s 5G 4G 3G 2G 1G 87%

Search by customer or order ID

All

Pending

In Progress

Completed

#ORD003

Amit Singh

In Progress

Total

₹570.00

11 items

30/7/2026

#ORD005

Vikram Verma

In Progress

Total

₹1210.00

11 items

30/7/2026

#ORD007

Karan Malhotra

In Progress

Total

₹940.00

7 items

30/7/2026

#ORD011

Suresh Reddy

In Progress

Total

9 items

Completed Orders Filter

5:02

1.83 MB/s

Search by customer or order ID

All

Pending

In Progress

Completed

#ORD001

Rahul Sharma

Completed

Total

₹710.00

9 items

30/7/2026

#ORD002

Priya Patel

Completed

Total

₹1010.00

7 items

30/7/2026

#ORD006

Neha Desai

Completed

Total

₹530.00

7 items

30/7/2026

#ORD009

Rohan Kapoor

Completed

Total

7 items

10. GitHub Repository & APK

Repository: <https://github.com/Sandesh4141/Restaurant-solution>

APK Release: <https://github.com/Sandesh4141/Restaurant-solution/releases>

Build Configuration: The project uses EAS Build (eas.json) with a dedicated 'release' profile configured for Android APK builds. The development and preview profiles support internal distribution, and the production profile uses autoIncrement for version management.

11. Conclusion

The Restaurant Order Management App successfully implements all assignment requirements: a secure login screen, a searchable and filterable orders list with 15 sample orders, detailed order views, and a status update workflow with local persistence. The project demonstrates a clean feature-based architecture, proper TypeScript typing throughout, and a well-structured state management approach using React Context and AsyncStorage. Key learnings include the importance of memoizing context values to prevent unnecessary re-renders, combining search and filter logic in a single useMemo for performance, and handling AsyncStorage seeding for first-launch scenarios. The app is production-ready with EAS Build configured for APK releases.